Teaching Statement

Michael Lehn, Ph.D. · Senior Lecturer, Institute of Numerical Mathematics, Ulm University

THE PROBLEM BEFORE THE SOLUTION

One principle shapes almost everything I do as a teacher: I do not explain a solution to a problem that students do not yet understand. Before I introduce a method, a data structure, or a piece of hardware, I want students to see the problem it solves and why that problem matters. Once the problem is clear, the solution is no longer an arbitrary rule to be memorized; it becomes a natural next step that students can follow, question, and often anticipate. Teaching from first principles follows from this: if every new concept has to answer a question students already have, a course has to begin where the questions begin.

Since 2008 I have taught at Ulm University in three settings: required mathematics courses for science and engineering students, numerical mathematics, and computing courses, including a two-course sequence in high-performance computing that I designed.

SEEING THE WHOLE MACHINE: HPC0

Introduction to High Performance Computing (HPC0), an undergraduate elective, starts from a question that is easy to ask and hard to answer: what actually happens when a program runs? Bottom-up, students use the CircuitVerse simulator to build digital circuits from basic logic gates up to a small ALU of their own. They then explore processor design by developing and experimenting with an instruction set step by step, guided by videos and classroom exercises. For the rest of the course they use a provided instruction set with ulm-generator, a toolchain I developed, which supplies the processor components and generates an assembler, a simulator, and a debugger. Because students have built an ALU and experimented with an instruction set themselves, they can reason about how the remaining components could be implemented without constructing each of them from scratch. Top-down, they program in ABC, a small C-like language I developed for teaching, and implement the parts of a compiler as regular assignments, from symbol tables and a lexer to a parser and code generators. Toward the end of the semester the two strands meet: the compiler generates code for this target architecture.

The same principle governs individual topics. Finite state machines are not introduced as a definition: students first encounter state-dependent behavior in the modal text editor they use every day, then need exactly this idea to write a lexer, and only then meet the formal concept. HPC0 is taught as a flipped classroom. Students prepare with short videos, and class time is spent on worksheets, with a teaching assistant and me working with individual students until the exercises run. The worksheets, videos, and lecturer notes are openly available in English.

MEASURING BEFORE OPTIMIZING: HPC1

High Performance Computing 1 is a graduate course that I designed and lead; part of the teaching is delegated to a co-instructor. Its central problem is the gap between what the hardware can do and what straightforward code achieves. Students measure that gap themselves by comparing their own implementations with the theoretical peak performance of the machine, and the cache hierarchy, blocking, and packing are introduced as answers to a problem they have already seen in their own numbers. In a project-based progression, they build their own optimized matrix–matrix product, integrate and tune provided SIMD micro-kernels, and use their kernels in a blocked LU factorization. When it still lags behind a vendor library, the next step is to test a hypothesis about where the time is spent. Parallel programming follows the same pattern. In guided exercises, students analyze and repair deliberately simple worker and thread-pool prototypes built on C++ threads, for example by explaining why a program crashes when a worker is destroyed or why all jobs end up on a single thread. They then use the thread pool from the course library to parallelize matrix computations, before moving on to OpenMP, MPI, and GPU computing with CUDA. The session materials are public.

HARDWARE AND SOFTWARE TOGETHER

Application-Oriented Software Development is a required undergraduate lab for students of electrical and biomedical engineering. Students begin by typing a short machine-code program in Intel HEX format into a text editor, flashing it onto a microcontroller, and observing the signal on a pin with an oscilloscope. They then move to assembly, C, and the Arduino framework, still working in the terminal with Makefiles, and build controllers on breadboards with potentiometers and eventually an inertial measurement unit. The controllers drive games that the students write in Python using object-oriented design.

MATHEMATICS FOR DIFFERENT AUDIENCES

Higher Mathematics 1–3 is a required sequence for students of physics, engineering, and computational science, for whom mathematics is a tool rather than their major. Here I teach traditional lectures, and the problem-first principle means connecting each topic to the questions it answers, both within mathematics and in the students' own fields. In numerical mathematics, from the undergraduate course in numerical linear algebra to graduate electives, I want students to understand why a method is needed before they derive it. My lecture notes for Numerical Linear Algebra follow this pattern. Before any theory of floating-point arithmetic, they ask why one should not simply solve problems exactly. They answer with a small experiment: two formulas for $\sqrt{x+1} - \sqrt{x}$ that are identical in exact arithmetic, one of which retains only about six correct digits in double precision for $x = 10^{10}$. Only then do the notes develop machine numbers and rounding-error analysis, and return to explain the result as cancellation. Later chapters use the same approach: the observation that even a simple 2×2 matrix has no LU factorization motivates pivoting, and the slow convergence of the gradient method for ill-conditioned problems motivates the conjugate gradient method.
I also teach *Mathematical Software*, a required programming lab for future secondary-school mathematics teachers.

EVIDENCE AND REFLECTION

The student evaluations available to me cover 30 offerings of my courses since 2008. The median overall rating is 5.2 on a six-point scale, with 6 as the best value, and most reports that include a comparison place my course above the average of mathematics lectures. I treat these results as indications rather than measurements, since recent courses had few respondents and not every offering produces a report. The criticisms are at least as useful, and they are consistent: in large lectures, students ask for a slower pace and for complete written lecture notes, and in the computing courses they point to a heavy workload. Balancing ambitious projects against a realistic workload is the tension I think about most when I plan a course, because the projects students value most are also the ones that cost them the most time.

MENTORING AND SERVICE

I have supervised about 45 bachelor's and master's theses since 2008, mostly in numerical mathematics and high-performance computing. Outside the university, I have mentored projects in Jugend forscht, Germany's national youth science competition, since 2008, and I co-direct the Ulm location of the Schülerforschungszentrum Südwürttemberg, a student research center where I teach extracurricular courses. Since 2002 I have regularly been elected as an academic staff representative to the study commission for the mathematics degree programs.

For a teaching-focused position beginning in Fall 2027, I would bring experience with large service courses, numerical mathematics at several levels, and computing courses built from the ground up. In each setting my aim is the same: students should understand the problem well enough that the solution makes sense before I present it.